

# Redes Neurais do tipo Autoencoders

Jônatas Rafael

May 2026

## 1 Autoencoders e aprendizado não supervisionado

Redes Neurais do tipo autoencoders atuam dentro do paradigma de aprendizado não supervisionado. Esse tipo de aprendizado de máquina é aquele em que não se busca treinar redes para realizar previsões, mas sim para realizar outras operações, como agrupamentos (*Clusterização*) ou redução de dimensionalidade, por exemplo.

O que acontece é que não se têm um *target* como no aprendizado supervisionado, mas o *target* é o próprio input. O que o Autoencoder tem como objetivo é reproduzir o dado de entrada sem "copiar" (pois apenas copiar os dados não seria útil), mas aprendendo as propriedades intrínsecas dos dados. [1]

## 2 Estrutura do Autoencoder

Por simplicidade, abordaremos a estrutura do autoencoder simétrico. Dessa estrutura fazem parte o encoder e o decoder. O objetivo do encoder é comprimir os dados, e o do decoder é reconstruir os dados a partir da saída gerada pelo encoder (*bottleneck*).

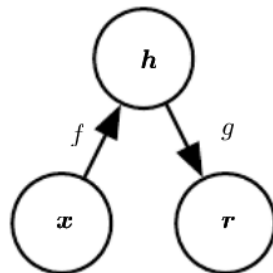


Figura 1: Encoder e decoder[1]

Matematicamente, teríamos:

- Para o encoder:  $h = f(x)$
- Para o decoder:  $r = g(h)$
- E gostaríamos de obter algo semelhante a  $g(f(x)) = x$

Dessa forma, se os dados tem, por exemplo,  $N$  features, o encoder tem na sua primeira camada  $N$  neurônios. Então, a estrutura do encoder é composta de camadas que reduzem o número de neurônios progressivamente, até chegar no *bottleneck* (ou gargalo, em português, ou simplesmente  $h$ , como na figura 1). O *bottleneck* é definido como o espaço intermediário ao encoder e decoder, e tem o menor número de dimensões do que as camadas tem neurônios. É aí que está o chamado **espaço latente**, o espaço onde a informação está comprimida.

Em seguida, temos o decoder. Sua estrutura é inversa à do encoder. Sua primeira camada recebe os dados do espaço latente, e o número de neurônios vai aumentando progressivamente até chegar ao número de neurônios  $N$  que a primeira camada do encoder tinha [2]. Temos então, que, de forma geral, um Autoencoder é "composto de duas **MLPs**" com funções distintas, o encoder e o decoder.

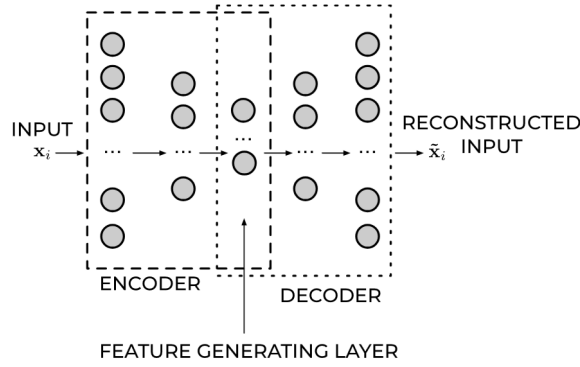


Figura 2: Estrutura do Autoencoder. Aqui o *bottleneck* é o *Feature generating layer*[2]

Entre as camadas, diversas funções de ativação podem ser aplicadas. Dentre elas, temos a ReLU (que foi aplicada nos notebooks de implementação didática em Pytorch e no estudo que compara PCA e Autoencoders para redução de dimensionalidade, disponíveis nesse repositório) e a Sigmoid, por exemplo.

### 3 Autoencoders incompletos e supercompletos

Esse tópico se refere a como  $h$  é com relação a  $x$ , e está associado a como se pretende que o modelo absorva a informação da entrada. Se o número de dimensões de  $h$  é menor que de  $x$ , então o autoencoder é incompleto. Reduzir a dimensão de  $h$  com relação a  $x$  é uma forma de forçar o modelo a obter propriedades relevantes da entrada sem copiá-la. A ideia é minimizar uma função de erro

$$L(x, g(f(X))) \quad (1)$$

em que se o autoencoder é composto por funções de ativação lineares e  $L$  é o erro quadrático médio, temos um PCA. Entretanto,  $h$  pode ter dimensão maior que  $x$  (caso em que o autoencoder é supercompleto), e mesmo que o encoder e o decoder sejam não lineares, é possível que o autoencoder ainda detecte os padrões relevantes dos dados, por meio de um mecanismo chamado de regularizador. Um regularizador em Autoencoder é basicamente um mecanismo que gera esparsidade no espaço latente, por meio de regularização  $l1$  ou  $l2$ , por exemplo [2].

### 4 Treino de Autoencoders

O treino de Autoencoders segue a mesma linha do treino das Redes Neurais MLP. Um número de épocas é definido, em que os dados passam pelo encoder e pelo decoder. A seguir, a loss é calculada com base na reconstrução do decoder e do respectivo dado original. Então, os gradientes são zerados e depois calculados (backpropagation), os pesos ajustados e uma nova época começa.

## 5 Aplicações de Autoencoders

### 5.1 Redução de dimensionalidade

Como mencionado antes, quando  $h$  tem menos dimensões do que  $x$  (chamado de método de gargalo), o encoder naturalmente comprimiu os dados de entrada em um número menor de dimensões. Isso é muito útil em machine learning, por que reduz o custo computacional de treino e evita a maldição da dimensionalidade (dados ficam esparsos e modelos têm seu desempenho piorado). Além disso, há vantagens em utilizar autoencoders com relação à PCA (um dos métodos mais clássicos de redução de dimensionalidade): em primeiro lugar, autoencoders são capazes de redução de dimensionalidade não linear. Além disso, autoencoders são computacionalmente mais eficientes nessa tarefa, por que para fazê-la não são necessários todos os dados, mas apenas uma parte, já que o modelo aprende o padrão dos dados. Em PCA, para a redução de dimensionalidade, todo o dataset é necessário, o que implica que quando se tem um aumento muito grande no volume de dados, PCA pode ficar impraticável. [2]

A redução de dimensionalidade aplicada em problemas de classificação também é capaz de reduzir o tempo necessário para que o modelo realize o treinamento em ordens de magnitude, tendo perdas relativamente pequenas [2].

## 5.2 Detecção de anomalias

Anomalias (outliers, ou seja, dados muito distoantes do resto do dataset) podem ser encontradas utilizando autoencoders. Isso é feito calculando o erro de reconstrução (um erro que é calculado comparando o dado de saída com o de entrada, depois que os dados passam pelo modelo) para todos os dados, e depois *rankeando* eles pelos maiores erros de reconstrução [2]. Por exemplo, se estou trabalhando em um problema com imagens de ressonância magnética do tórax, posso descobrir se em meu conjunto de dados há outro tipo de imagem (que pode atrapalhar o desempenho do meu modelo), aplicando esse método. Uma aplicação interessante disso é detectar fraudes financeiras.

## 5.3 Retirar ruído de dados

Autoencoders podem ser utilizados para retirar ruídos de dados. Por exemplo, no conjunto de dados MNIST, se aplicado um ruído gaussiano que altera os valores numéricos do cinza dos pixels, o autoencoder é ainda assim capaz de reconstruir a imagem original (por "original" quero dizer, sem o ruído) com boa efetividade [2]

# 6 Análise dos resultados do estudo comparativo

Como mencionado, nesse repositório há um estudo que compara o desempenho de modelos de Machine Learning que receberam dados (com o objetivo de reduzir dimensionalidade) de PCA e de Autoencoders. Essa seção se dedica a discutir esses resultados. Esse estudo está mais detalhado no Jupyter Notebook que tem o nome "Autoencoders\_PCA\_estudo.ipynb".

O arquivo "Autoencoders\_PCA\_estudo.py" foi utilizado para plotar esses gráficos. A ideia é manter fixos a quantidade de features (100), o *bottleneck* (10) e variar a quantidade de features informativas e o nível de ruído adicionado aos dados. Os modelos de Machine Learning utilizados foram Floresta Aleatória, Support Vector Regressor e Regressão Linear.

Os resultados obtidos foram:

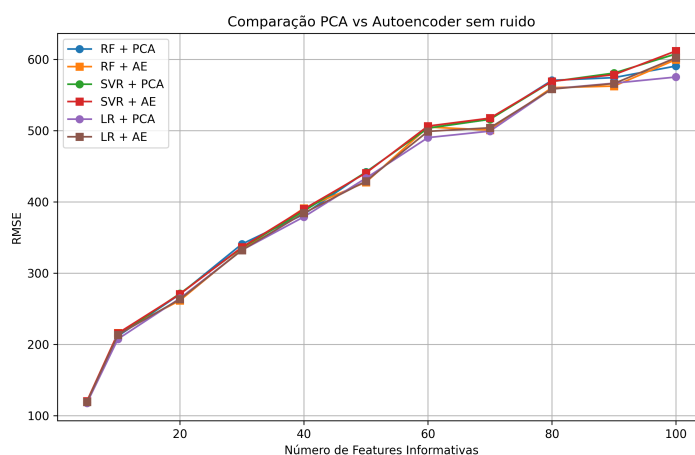


Figura 3: Desempenho dos modelos sem ruído

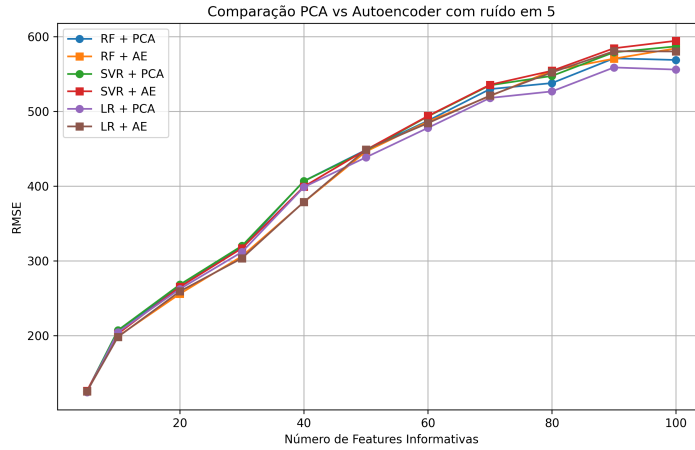


Figura 4: Desempenho dos modelos com ruído em 5

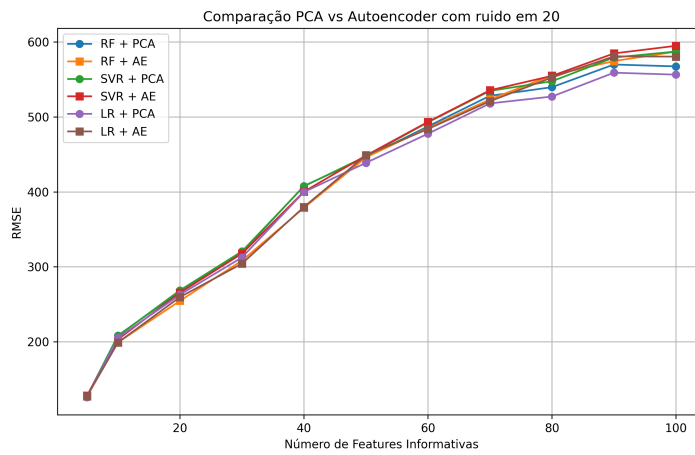


Figura 5: Desempenho dos modelos com ruído em 20

De forma superficial, o desempenho dos modelos é semelhante tanto comparando modelos diferentes quanto comparando PCA e Autoencoders como técnicas para redução de dimensionalidade. O comportamento geral é que o PCA produz um RMSE ligeiramente menor quando o número de features informativas se aproxima de 100. O único ponto em que o Autoencoder é um pouco melhor é para os casos com ruído quando o número de features informativas é de 40.

Esses resultados estão semelhantes aos resultados obtidos por [4], que fez uma comparação semelhante utilizando datasets como MNIST e CIFAR-10. O resultado nessa referência também foi um desempenho parecido entre PCA e Autoencoders. Isso chama atenção para o fato de que PCA permanece um método competitivo frente à técnicas mais sofisticadas, por ter uma implementação bem mais simples e menos custosa computacionalmente na maioria dos casos.

## 7 Referências do repositório

[1] Deep Learning GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. Deep learning. Cambridge: MIT Press, 2016. Disponível em: <http://www.deeplearningbook.org>. Acesso em: 20 abr. 2026.

[2] MICHELUCCI, Umberto. An introduction to autoencoders. 2022. Disponível em: <https://arxiv.org/abs/2201.03898>. Acesso em: 20 abr. 2026

[3] CASSAR, Daniel. ATP-203 4.0 - Árvore de decisão e floresta aleatória.ipynb. 2025. Material didático disponibilizado no Teams no curso de Machine Learning, Ilum Escola de Ciência, Campinas. . Acesso em: 09

mai. 2026.

[4] FOURNIER, Quentin; ALOISE, Daniel. Empirical comparison between autoencoders and traditional dimensionality reduction methods. 2021. Disponível em: <https://ieeexplore.ieee.org/abstract/document/8791727>. Acesso em 10. mai. 2026.

[5] CASSAR, Daniel. ATP-203 8.0 - Redução de Dimensionalidade com PCA.ipynb. 2025. Material didático disponibilizado no Teams no curso de Machine Learning, Ilum Escola de Ciência, Campinas. . Acesso em: 10 mai. 2026.

[6] IBM. What is dimensionality reduction? IBM, s.d. Disponível em: <https://www.ibm.com/topics/dimensionality-reduction> . Acesso em: 10 maio 2026.

[7] OPENAI. ChatGPT. 2026. Disponível em: <https://chat.openai.com/>. Acesso em: 10 mai. 2026.

Link para acesso às conversas:

<https://chatgpt.com/share/6a00d1a7-2e00-83e9-a492-698e39ae7d30>

<https://chatgpt.com/share/6a00da3d-15fc-83e9-a7aa-adfc9dc62f73>

[8] scikit-learn. Supervised learning. [S. l.], s.d. Disponível em: [https://scikit-learn.org/stable/supervised\\_learning.html](https://scikit-learn.org/stable/supervised_learning.html). Acesso em: 10 maio 2026.

[9] PyTorch. MNIST Dataset. Disponível em: <https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html>. Acesso em 10 mai. 2026.

[10] LOEBER, Patrick. Autoencoder In PyTorch - Theory & Implementation. YouTube, 22 mar. 2021. Disponível em: <https://www.youtube.com/watch?v=zp8clK9yCro> . Acesso em: 9 maio 2026.

[11] CASSAR, Daniel. ATP-303 NN 3.2 - Construindo e treinando redes neurais com PyTorch e Lightning.ipynb. 2026. Material didático disponibilizado no Teams do curso de Redes Neurais e Algoritmos Genéticos da Ilum Escola de Ciência, Campinas. Acesso em 10 mai. 2026.